

进程同步的算法题解法

1、明确三个基本概念

临界资源

同步

互斥

2、熟记利用信号量实现同步、互斥和控制资源访问的经典范式

3、熟悉课本上的经典同步问题的算法实现

4、分析进程同步和互斥问题的方法步骤

- 1) 关系分析。找出问题中的进程数，并且分析他们之间的同步和互斥关系。同步、互斥关系直接按照经典范式改写
- 2) 整理思路。找出解决问题的关键点，并且根据课本上的经典题目找出解决思路。根据进程的操作流程确定wait操作，signal操作的大致顺序。
- 3) 设置信号量。根据上面两步，设置需要的信号量，确定初值，完善整理。

■ 例1

问题描述：桌子上有一只盘子，每次只能向其中放入一个水果。爸爸专向盘子中放苹果，妈妈专向盘子中放橘子，儿子专等吃盘子中的橘子，女儿专等吃盘子中的苹果。只有盘子为空时，爸爸或妈妈可向盘子中放一个水果；仅当盘子中有自己需要的水果时，儿子或女儿可以从盘子中取出水果来吃。

Semaphore plate=1,apple=0,orange=0;

```
Dad(){  
    While(1){  
        Prepare an apple;  
        Wait(plate);  
        Put the apple on the plate;  
        Signal(apple);  
    }  
}
```

```
Mom(){  
While(1){  
    Prepare an orange;  
    Wait(plate);  
    Put the orange on the plate;  
    Signal(orange);  
}  
}
```

```
Son(){  
    While(1){  
        Wait(orange);  
        Take an orange from the plate;  
        Signal(plate);  
        Eat the orange;  
    }  
}
```

```
daughter(){  
    While(1){  
        Wait(apple);  
        Take an apple from the plate;  
        Signal(plate);  
        Eat the apple;  
    }  
}
```

例2

某寺庙，有小和尚，老和尚若干，有一水缸，由小和尚提入水缸供老和尚饮用。

水缸可容纳**10**桶水，水取自同一井中。水井窄，每次只能容一个桶取水，水桶总数为**3**个，每次入缸取水仅为**1**桶，且不可同时进行。

试着给出有关从缸取水，入水的算法描述。

- 从题中可以看出水井和水缸都是临界资源，引入 **well**, **vat**;
- 三个水桶无论从井中取水还是放入水缸中都只能是一次一个，给它们一个信号量 **pail** 表示空的水桶的数量，抢不到水桶的继续等待，水缸满时，不可以继续放水;
- 设置一个信号量 **empty** 表示水缸现在还能容纳多少水量，水缸空时，不可以继续取水;
- 设置一个信号量 **full** 表示水缸现在的容量。

- 本题设置5个信号量:
- Semaphore well=1; //互斥信号量, 表示水井互斥使用
- Semaphore vat = 1; //互斥信号量, 表示水缸互斥使用
- Semaphore pail = 3; //表示能用的水桶数量, 初始值为3
- Semaphore empty = 10; //表示水缸现在还能容纳多少水, 初始值为10, 水缸初始值为空
- Semaphore full = 0; //表示水缸现在的容量, 初始值为0, 水缸初始值为空

- //老和尚
- While(1){
- P(full) //申请空水缸里的水操作，有则继续，没有等待
- P(pail) //申请空桶操作，有则继续，没有等待
- P(vat); //占用水缸互斥资源
- 从水缸中取一桶水;
- V(vat); //取水后，放弃水缸这个互斥资源，
- V(empty); //取水后，水缸的剩余空间+1
- 喝水;
- V(pail); //用完桶后，空桶资源+1
- }

```
■ //小和尚
■ While(1){
■     P(empty); //观察水缸是否有剩余空间，有的话，才能去打
    水
■     P(pail); //申请空桶资源
■     P(well); //打水的时候，水井互斥使用
■     打水;
■     V(well); //释放水井互斥资源
■     P(vat); //把水放进水缸，申请，水缸互斥资源
■     倒水;
■     V(vat); //倒完水后，释放水缸互斥资源
■     V(full); //倒完水后，水缸先有的水容量+1
■     V(pail); //空桶资源+1
■ }
```

■ 例3

■ 问题描述:假设有三个抽烟者和一个供应者。每个抽烟者不停地卷烟并抽掉它，但是要卷起并抽掉一支烟，抽烟者需要三种材料：烟草、纸和胶水。三个抽烟者中，第一个拥有烟草、第二个拥有纸、第三个拥有胶水。供应者无限地提供这三种材料，每次将两种不同的材料放到桌子上，拥有剩下那种材料的抽烟者将会卷一根烟并抽掉它，并给供应者一个信号告诉完成了，供应者就会继续放两种材料在桌上，以让一位抽烟者抽烟，这种过程一直重复下去。

- `Int random;` `//存储随机数`
- `Semaphore offer1=0;` `//offer1对应烟草和纸组合`
的资源
- `Semaphore offer2=0;` `//offer2对应烟草和胶水组`
合的资源
- `Semaphore offer3=0;` `//offer3对应纸和胶水组合`
的资源
- `Semaphore finish=0;` `// 表示抽烟是否完成`

```

■ Offer(){
                                                    //提供者
■ While(1){
■     Random=随机生成的一个整数;
■     Random=random%3;
■     If(random==0) Signal(offer1);
■     If(random==1) Signal(offer2);
■     If(random==2) Signal(offer3);
■     Wait(finish);
■ }
■ }
```

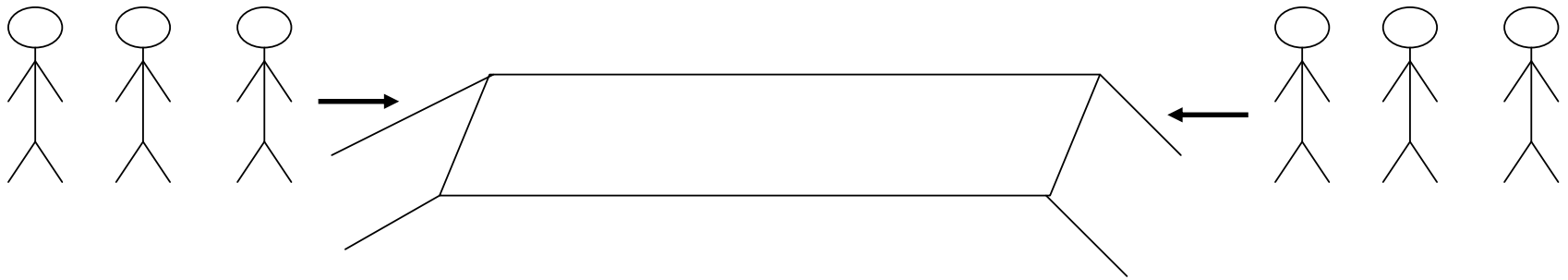
```
■ tobacco (){                                //拥有烟草的吸烟者
■   while(1){
■       wait(offer3);
■       拿纸和胶水， 卷成烟， 抽掉;
■       Signal(finish);
■   }
■ }
```



```
■ paper (){                                     //拥有纸的吸烟者
■   while(1){
■     wait(offer2);
■     拿烟草和胶水， 卷成烟， 抽掉;
■     Signal(finish);
■   }
■ }
■
```

```
■ glue (){                                     //拥有胶水的吸烟者
■     while(1){
■     wait(offer1);
■     拿纸和烟草，卷成烟，抽掉;
■     Signal(finish);
■     }
■ }
```

- 例4
- As illustrated in the figure, on the two sides of a one-plank bridge(独木桥), there are two groups of soldiers that are composed of m and n people respectively and need to cross the bridge, but the narrow bridge allows only one group of the soldiers in the same direction to cross at the same time.
- One group of the soldiers is permitted to cross as long as there are no people on the bridge. Once one group of the soldiers begins walking on the bridge, the other group should be waiting to start crossing until all members of the first group have passed the bridge.



- Please design two semaphore-based processes to describe the crossing actions of the soldiers in the two groups. It is required
- (1) to define the semaphores and variables needed, explain their roles? and give their initial values; and
- (2) to illustrate the structures of processes for the soldiers in each group.

■ Answers:

- 该问题可归结为读写者问题，但2队士兵均为读者。
- （1）定义2个整型变量count1、count2，分别用于计数每组士兵过桥人数。
- 定义2个二元信号量mutex1、mutex2，用于控制对count1、count2的互斥访问。
- 定义二元信号量bridge，用于2队士兵间的互斥。
- 初始化：
 - count1=0，count2=0；
 - mutex1=1、mutex2=1；
 - bridge=1；

```
■ P1()  
■ {  
■     wait(mutex1);  
■     count1 ++;  
■     if count1 == 1  
■         wait(bridge); // 该组第1个士兵过桥时，阻塞另一组过桥  
■     signal(mutex1);  
■         .....  
■     crossing the bridge;  
■         .....  
■     wait(mutex1);  
■     count1 --;  
■     if count1 == 0  
■         signal(bridge); // 该组最后1个士兵过桥后，允许另一组过桥  
■     signal(mutex1);  
■ }.
```

```
■ P2()
■ {
■     wait(mutex2);
■     count2 ++;
■     if count2 == 1
■         wait(bridge); // 该组第1个士兵过桥时，阻塞另一组过桥
■     signal(mutex2);
■     .....
■     crossing the bridge;
■     .....
■     wait(mutex2);
■     count2 --;
■     if count2 == 0
■         signal(bridge); // 该组最后1个士兵过桥后，允许另一组过桥
■     signal(mutex2);
■ }.
```


Chapter 7: Deadlocks

- The Deadlock Problem
- System Model
- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock

The Deadlock Problem

- A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set.
- Example
 - ☞ System has 2 tape drives.
 - ☞ P_1 and P_2 each hold one tape drive and each needs another one.

The Deadlock Problem

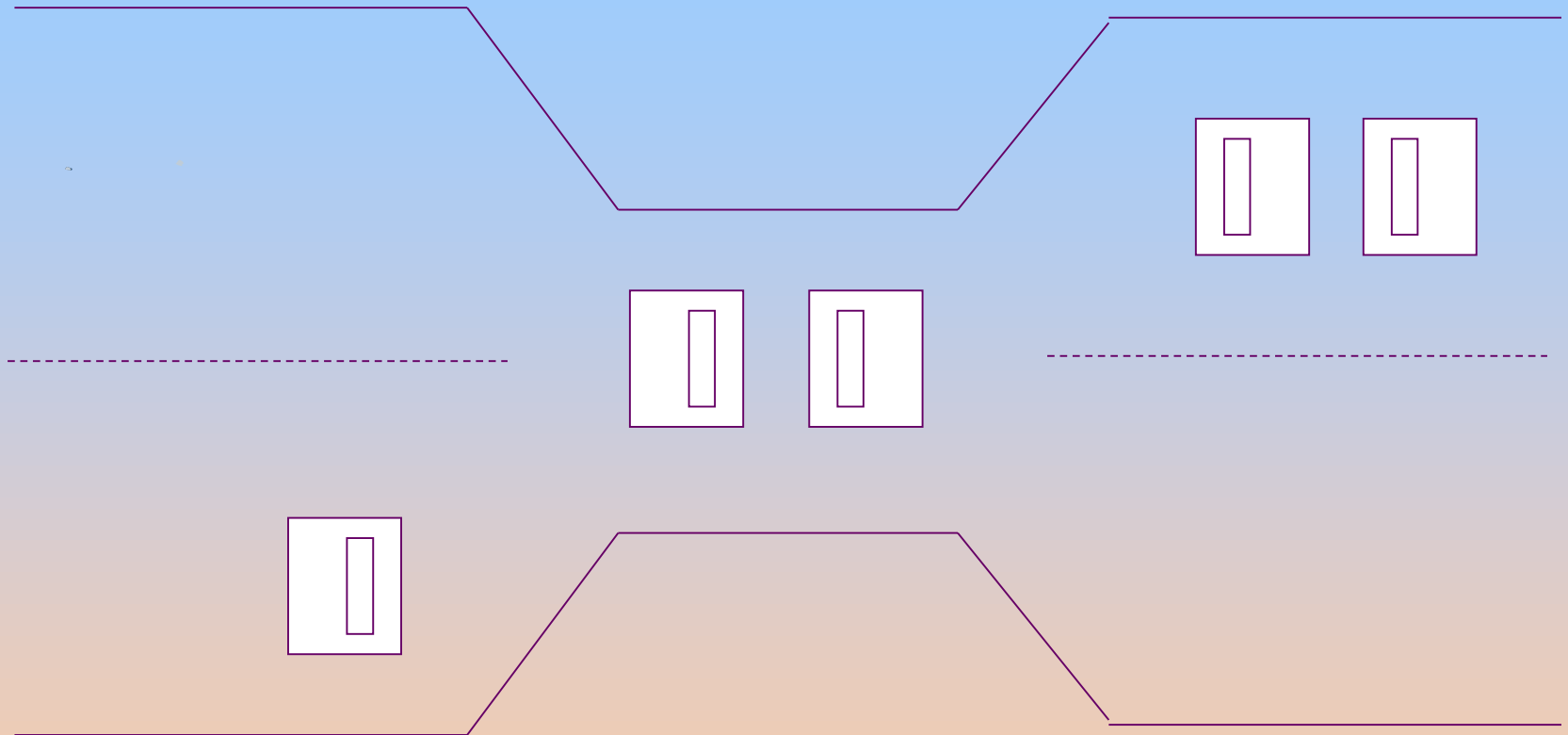
- A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set.
- Example
 - ☞ System has 2 tape drives.
 - ☞ P_1 and P_2 each hold one tape drive and each needs another one.

■ Example

☞ semaphores A and B , initialized to 1

P_0	P_1
<i>wait (A);</i>	<i>wait(B)</i>
<i>wait (B);</i>	<i>wait(A)</i>

Bridge Crossing Example



- Traffic only in one direction.
- Each section of a bridge can be viewed as a resource.
- If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rollback).
- Several cars may have to be backed up if a deadlock occurs.
- Starvation is possible.

System Model

- Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices
- Each resource type R_i has W_i instances.
- Each process utilizes a resource as follows:
 - 👉 request
 - 👉 use
 - 👉 release

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource.
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes.

- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_{n-1}\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_0 , and P_0 is waiting for a resource that is held by P_0 .

Resource-Allocation Graph

A set of vertices V and a set of edges E .

■ V is partitioned into two types:

☞ $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system.

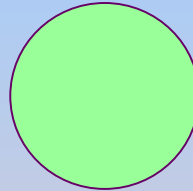
$R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system.

■ request edge – directed edge $P_1 \rightarrow R_j$

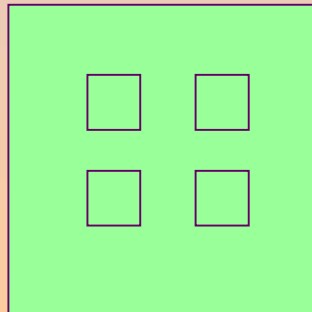
■ assignment edge – directed edge $R_j \rightarrow P_i$

Resource-Allocation Graph (Cont.)

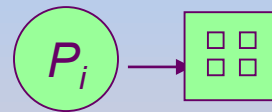
■ Process



■ Resource Type with 4 instances

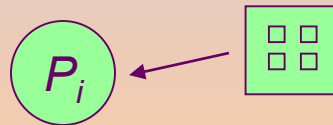


- P_i requests instance of R_j



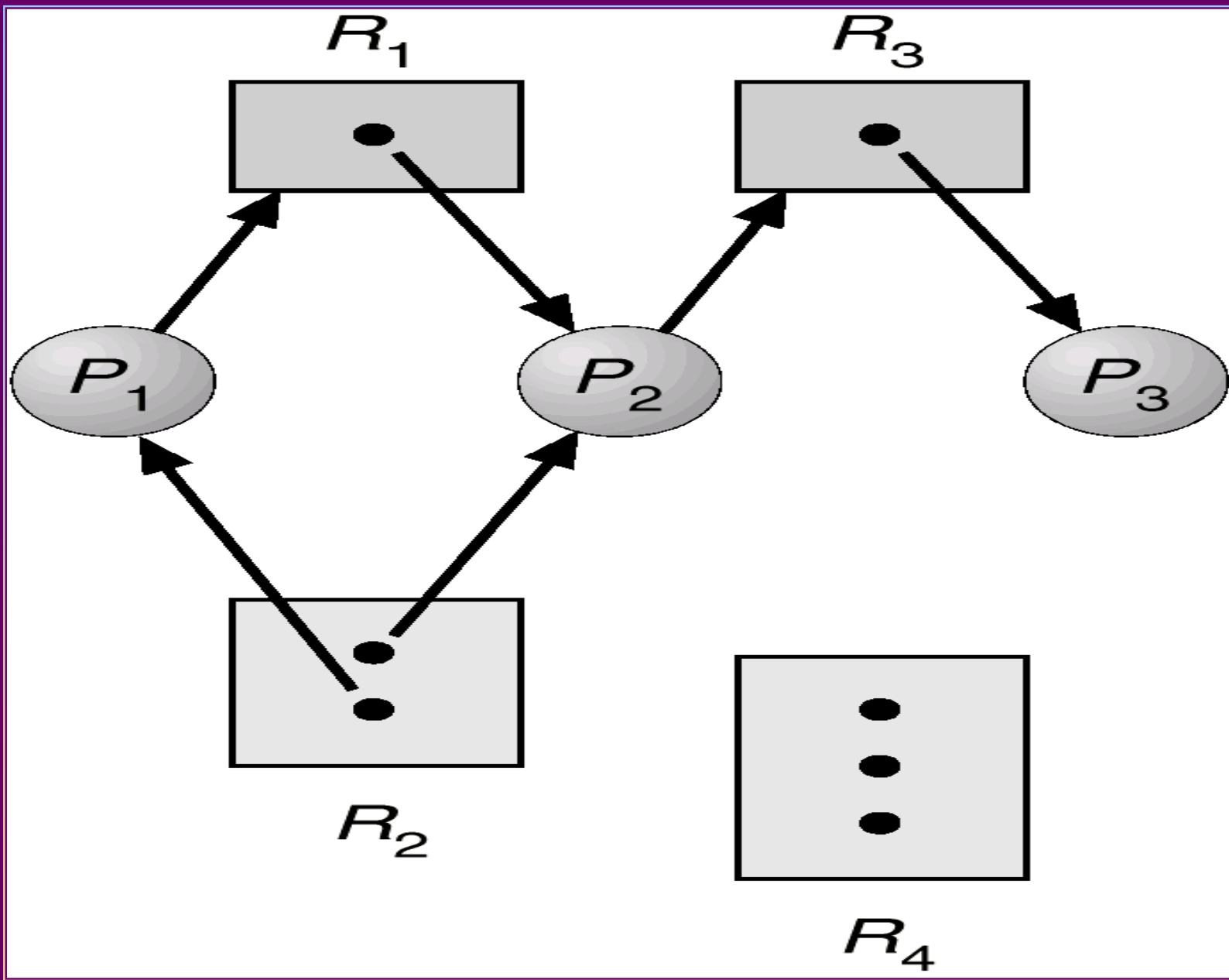
R_j

- P_i is holding an instance of R_j

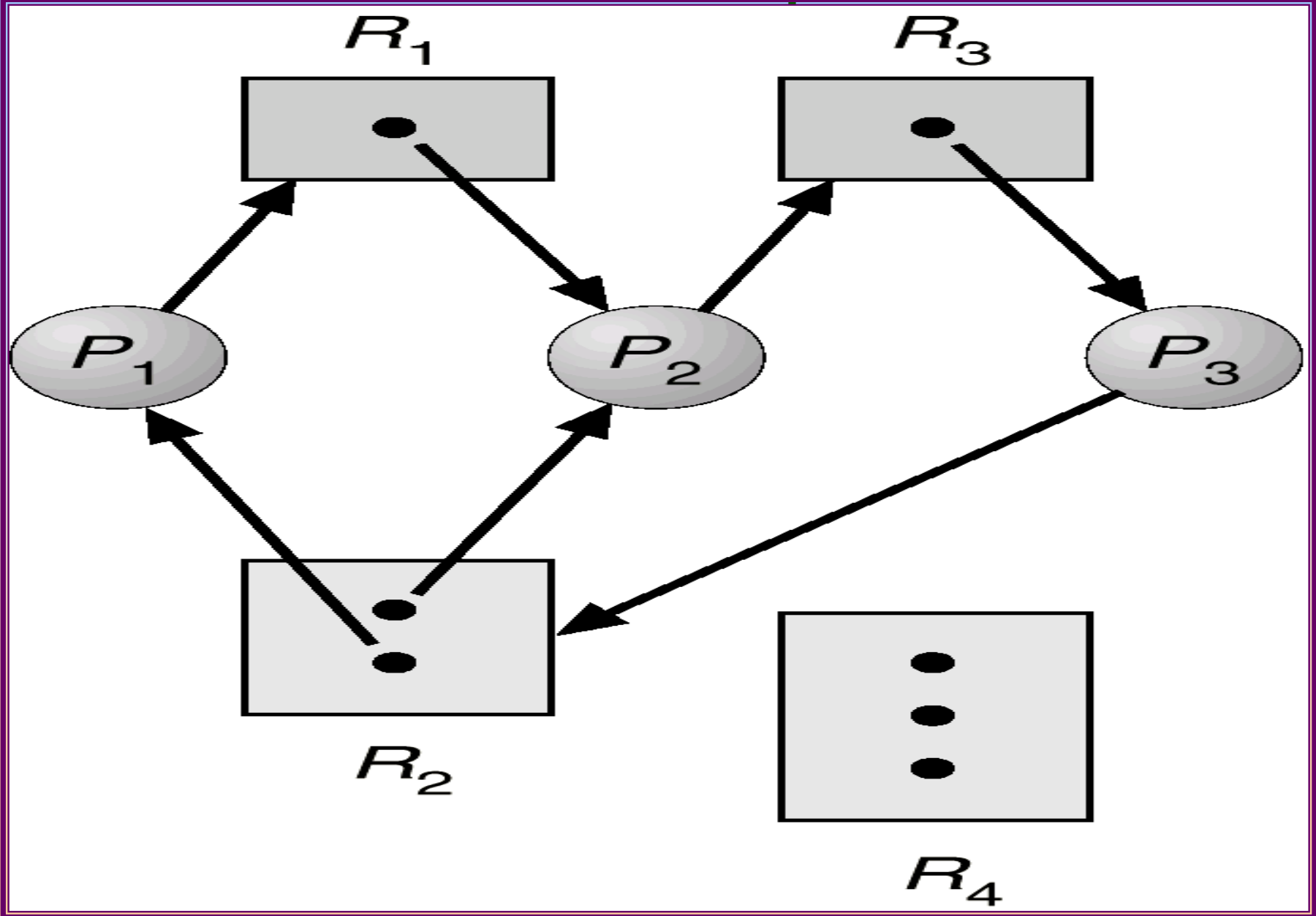


R_j

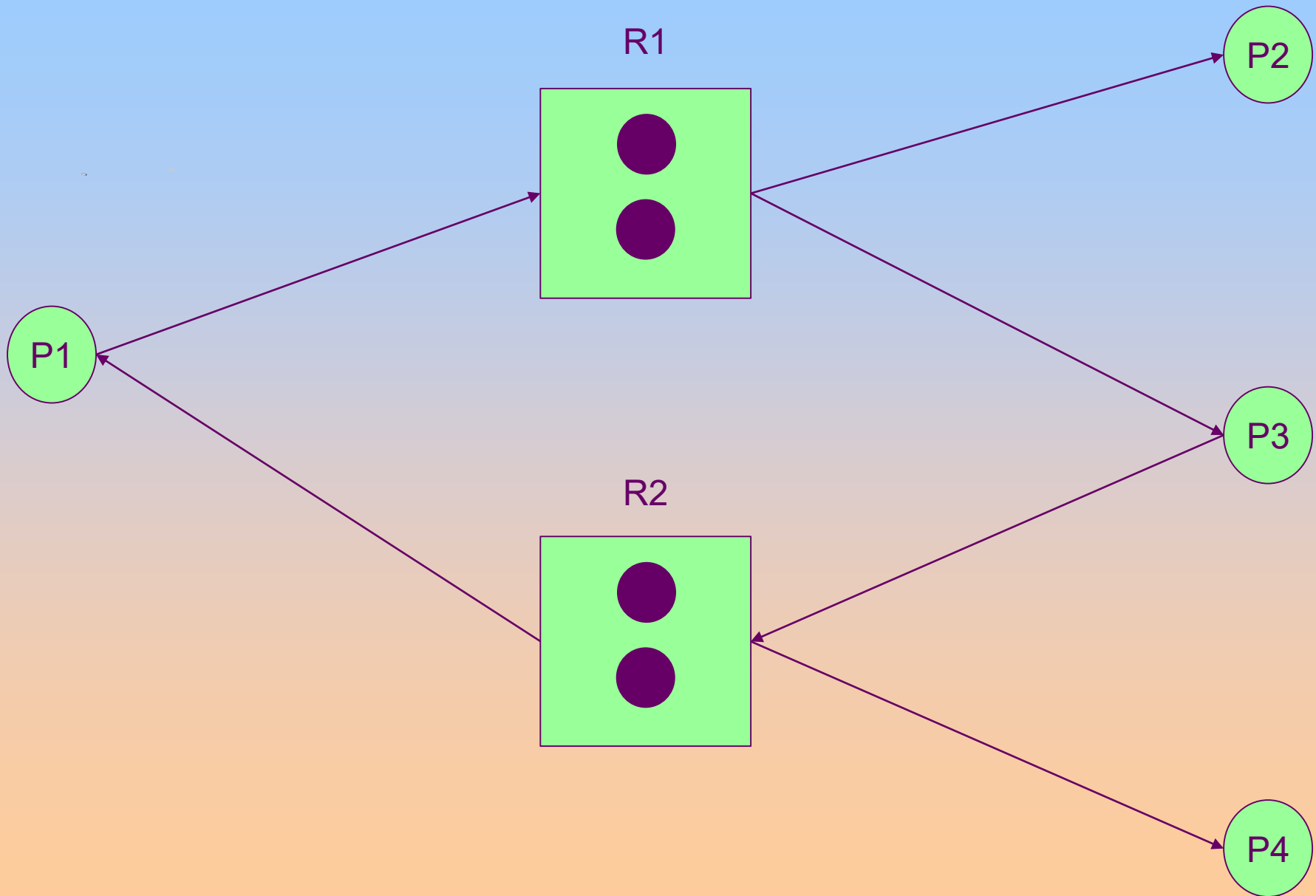
Example of a Resource Allocation Graph



Resource Allocation Graph With A Deadlock



Resource Allocation Graph With A Cycle But No Deadlock



Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock.
- If graph contains a cycle $\Rightarrow$
 - ☞ if only one instance per resource type, then deadlock.
 - ☞ if several instances per resource type, possibility of deadlock.

Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state.
- Allow the system to enter a deadlock state and then recover.
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX.

Deadlock Prevention

Restrain the ways request can be made.

- **Mutual Exclusion** – not required for sharable resources; must hold for nonsharable resources.

■ **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources.

☞ Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none.

☞ Low resource utilization; starvation possible.

Deadlock Prevention (Cont.)

■ No Preemption –

- ☞ If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released.
- ☞ Preempted resources are added to the list of resources for which the process is waiting.

☞ Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.

■ **Circular Wait** – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration.

Deadlock Avoidance

Requires that the system has some additional *a priori* information available.

- Simplest and most useful model requires that each process declare the *maximum number* of resources of each type that it may need.

- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes.

Safe State

- When a process requests an available resource, system must decide if immediate allocation leaves the system in a *safe state*.
- System is in safe state if there exists a safe sequence of all processes.

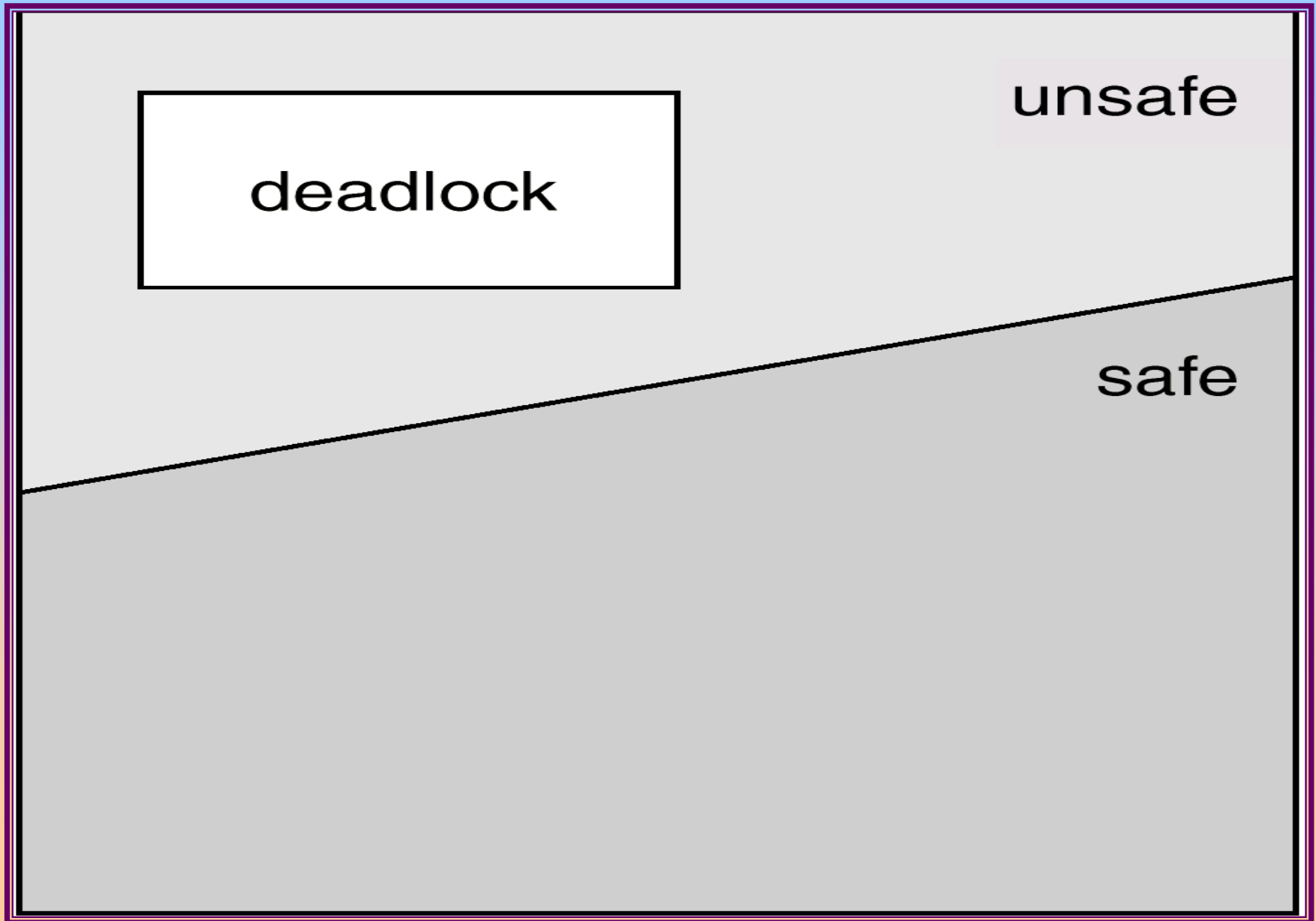
Sequence $\langle P_1, P_2, \dots, P_n \rangle$ is safe if for each P_i , the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j , with $j < i$.

- ➡ If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished.
- ➡ When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate.
- ➡ When P_i terminates, P_{i+1} can obtain its needed resources, and so on.

Basic Facts

- If a system is in safe state $\Rightarrow$ no deadlocks.
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock.
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.

Safe, Unsafe , Deadlock State

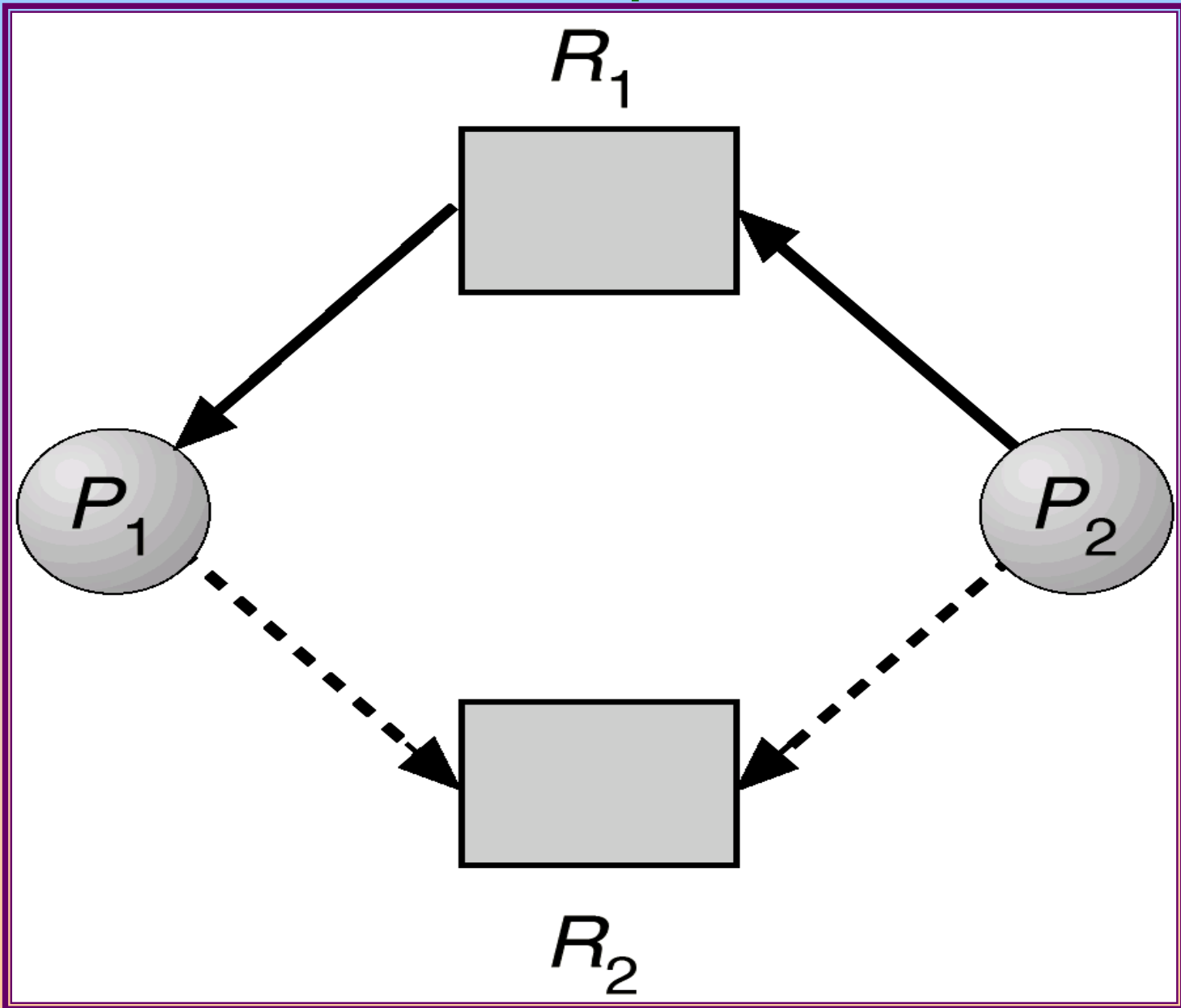


Resource-Allocation Graph Algorithm

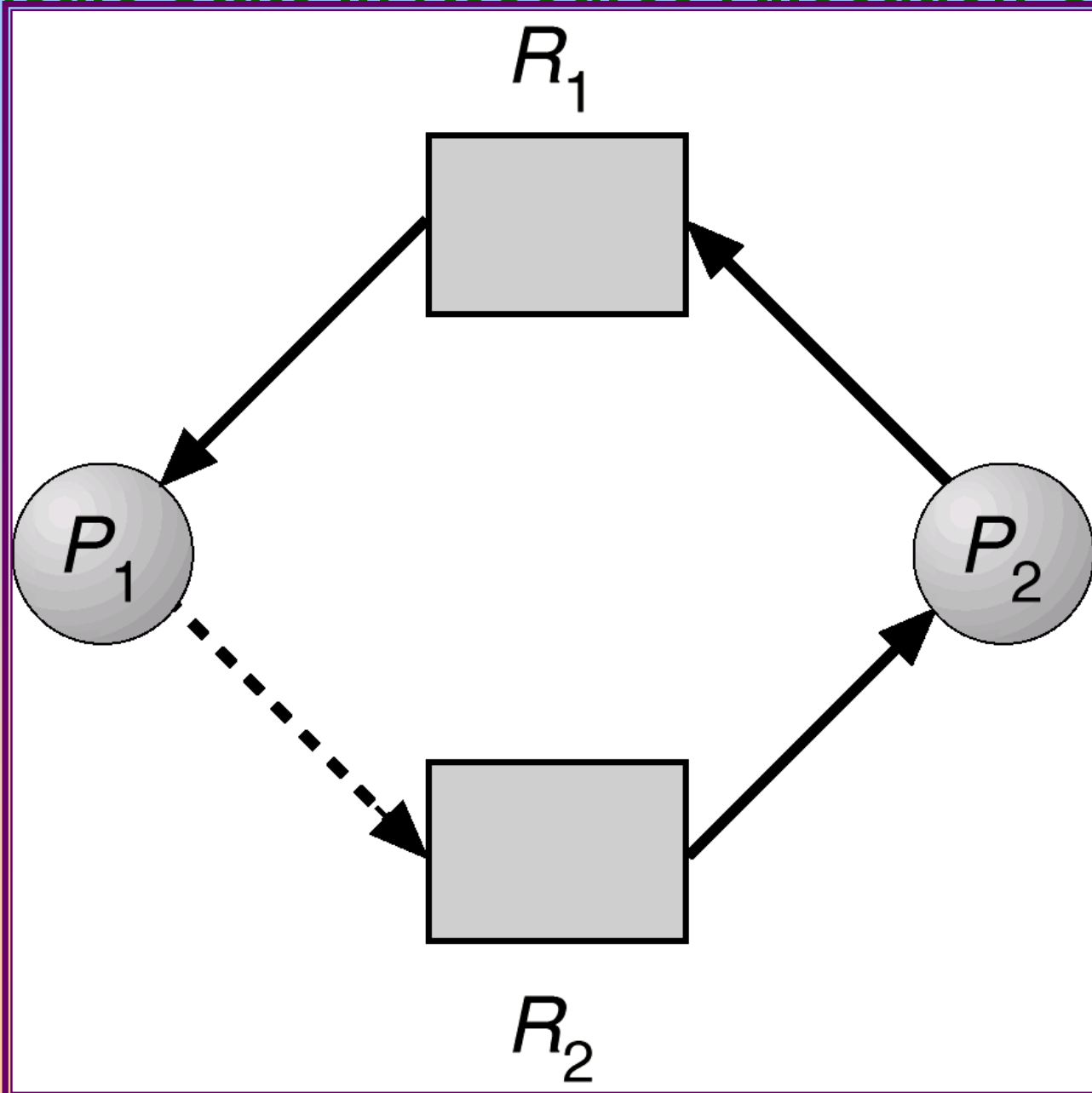
- *Claim edge* $P_i \rightarrow R_j$ indicated that process P_j may request resource R_j ; represented by a dashed line.
- Claim edge converts to request edge when a process requests a resource.

- When a resource is released by a process, assignment edge reconverts to a claim edge.
- Resources must be claimed *a priori* in the system.

Resource-Allocation Graph For Deadlock Avoidance



Unsafe State In Resource-Allocation Graph



Banker's Algorithm

- Multiple instances.
- Each process must a priori claim maximum use.

- When a process requests a resource it may have to wait.
- When a process gets all its resources it must return them in a finite amount of time.

Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- *Available*: Vector of length m . If $\text{available}[j] = k$, there are k instances of resource type R_j available.
- *Max*: $n \times m$ matrix. If $\text{Max}[i,j] = k$, then process P_i may request at most k instances of resource type R_j .

- *Allocation*: $n \times m$ matrix. If $\text{Allocation}[i,j] = k$ then P_i is currently allocated k instances of R_j .
- *Need*: $n \times m$ matrix. If $\text{Need}[i,j] = k$, then P_i may need k more instances of R_j to complete its task.

$$\text{Need}[i,j] = \text{Max}[i,j] - \text{Allocation}[i,j].$$

Safety Algorithm

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively. Initialize:

Work = *Available*

Finish [*i*] = *false* for *i* = 1, 3, ..., *n*.

2. Find and *i* such that both:

(a) *Finish* [*i*] = *false*

(b) $Need_i \leq Work$

If no such *i* exists, go to step 4.

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2.

4. If $Finish[i] == true$ for all i , then the system is in a safe state.

Resource-Request Algorithm for Process P_i

Request = request vector for process P_i . If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j .

1. If $Request_i \leq Need_i$ go to step 2.
Otherwise, raise error condition, since process has exceeded its maximum claim.
2. If $Request_i \leq Available$, go to step 3.
Otherwise P_i must wait, since resources are not available.

3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$Available = Available - Request_i;$

$Allocation_i = Allocation_i + Request_i;$

$Need_i = Need_i - Request_i;$

- *If safe $\Rightarrow$ the resources are allocated to P_i .*
- *If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored*

Example of Banker's Algorithm

- 5 processes P_0 through P_4 ; 3 resource types A (10 instances), B (5 instances), and C (7 instances).
- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Example (Cont.)

- The content of the matrix. Need is defined to be Max – Allocation.

	<u>Need</u>		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria.

Example P_1 Request (1,0,2) (Cont.)

- Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow \text{true}$.

	<u>Allocation</u>			<u>Need</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	4	3	2	3	0
P_1	3	0	2	0	2	0			
P_2	3	0	1	6	0	0			
P_3	2	1	1	0	1	1			
P_4	0	0	2	4	3	1			

- Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement.
- Can request for (3,3,0) by P_4 be granted?
- Can request for (0,2,0) by P_0 be granted?

Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

Single Instance of Each Resource Type

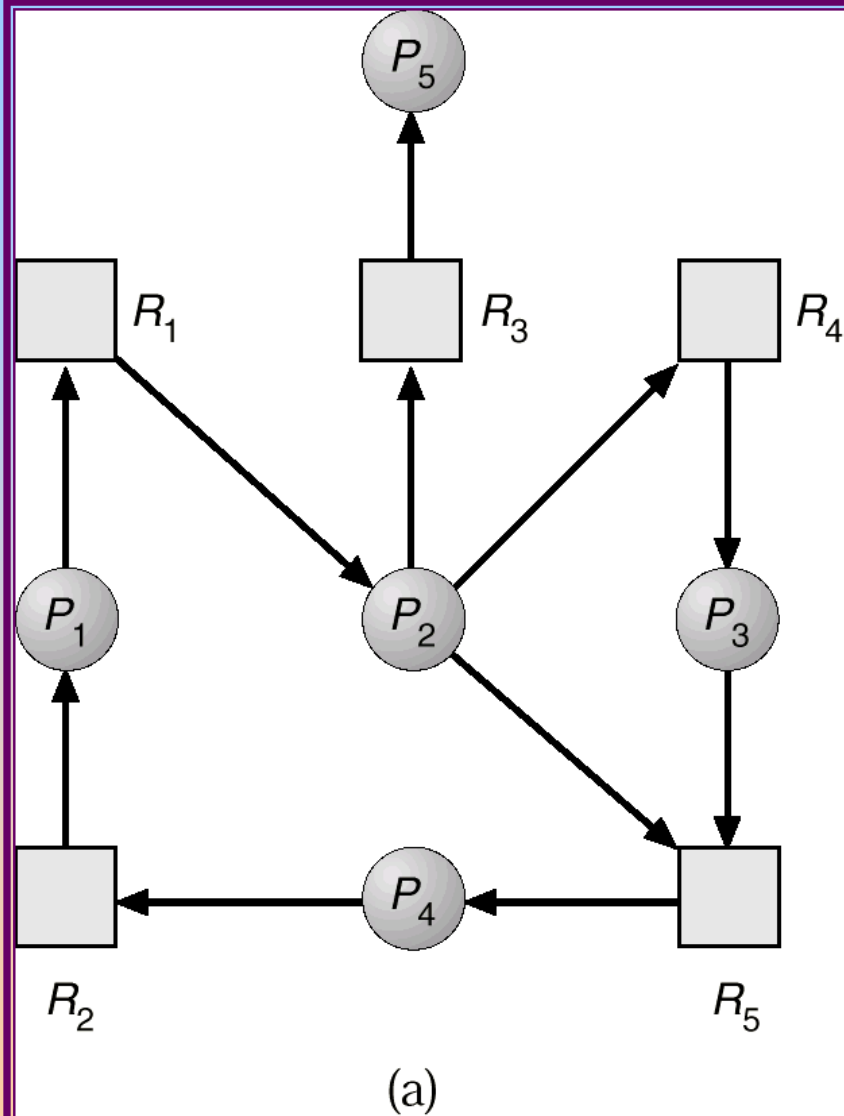
■ Maintain *wait-for* graph

☞ Nodes are processes.

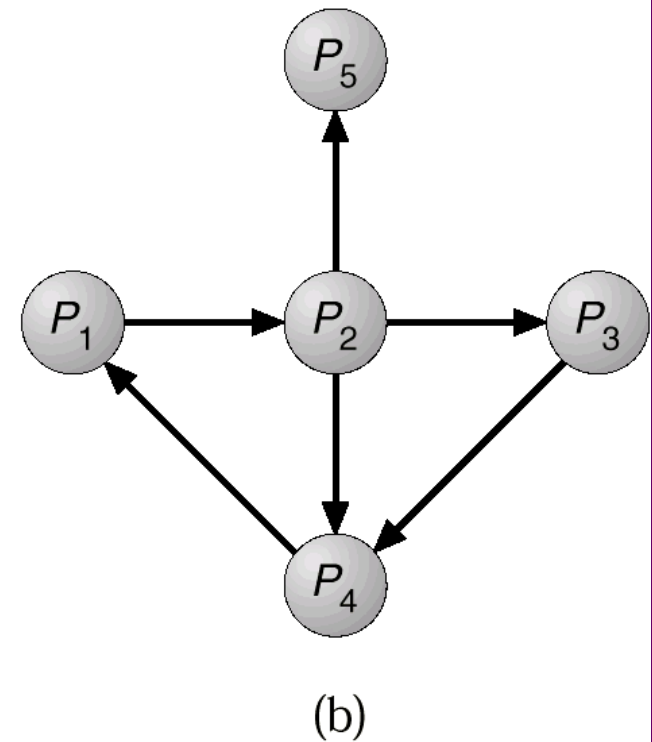
☞ $P_i \rightarrow P_j$ if P_i is waiting for P_j .

- Periodically invoke an algorithm that searches for a cycle in the graph.
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph.

Resource-Allocation Graph and Wait-for Graph



Resource-Allocation Graph



Corresponding wait-for graph

Several Instances of a Resource Type

- *Available:* A vector of length m indicates the number of available resources of each type.
- *Allocation:* An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.

- *Request:* An $n \times m$ matrix indicates the current request of each process. If $Request[i_j] = k$, then process P_i is requesting k more instances of resource type R_j .

Detection Algorithm

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively Initialize:
 - (a) *Work* = *Available*
 - (b) For $i = 1, 2, \dots, n$, if $Allocation_i \neq 0$, then
Finish[*i*] = false; otherwise, *Finish*[*i*] = true.

2. Find an index i such that both:

(a) $Finish[i] == false$

(b) $Request_i \leq Work$

If no such i exists, go to step 4.

Detection Algorithm (Cont.)

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2.
4. If $Finish[i] == false$, for some i , $1 \leq i \leq n$, then the system is in deadlock state.
Moreover, if $Finish[i] == false$, then P_i is deadlocked.

- Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state.

Example of Detection Algorithm

- Five processes P_0 through P_4 ; three resource types
A (7 instances), B (2 instances), and C (6 instances).
- Snapshot at time T_0 :

	<u>Allocation</u>			<u>Request</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	0			
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			

- Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in $Finish[i] = \text{true}$ for all i .

Example (Cont.)

P_2 requests an additional instance of type C.

Request

	A	B	C
P_0	0	0	0
P_1	2	0	1
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

■ State of system?

- ☞ Can reclaim resources held by process P_0 , but insufficient resources to fulfill other processes; requests.
- ☞ Deadlock exists, consisting of processes P_1 , P_2 , P_3 , and P_4 .

Detection-Algorithm Usage

- When, and how often, to invoke depends on:
 - ☞ How often a deadlock is likely to occur?
 - ☞ How many processes will need to be rolled back?
 - 📄 one for each disjoint cycle

- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.

Recovery from Deadlock: Process Termination

- Abort all deadlocked processes.
- Abort one process at a time until the deadlock cycle is eliminated.

- In which order should we choose to abort?
 - ➡ Priority of the process.
 - ➡ How long process has computed, and how much longer to completion.
 - ➡ Resources the process has used.
 - ➡ Resources process needs to complete.
 - ➡ How many processes will need to be terminated.
 - ➡ Is process interactive or batch?

Recovery from Deadlock: Resource Preemption

- Selecting a victim – minimize cost.
- Rollback – return to some safe state, restart process for that state.
- Starvation – same process may always be picked as victim, include number of rollback in cost factor.

Combined Approach to Deadlock Handling

■ Combine the three basic approaches

☞ prevention

☞ avoidance

☞ detection

allowing the use of the optimal approach for each of resources in the system.

- Partition resources into hierarchically ordered classes.
- Use most appropriate technique for handling deadlocks within each class.

Traffic Deadlock for Exercise 8.4

